Control Panel

Select Model Engine

gemini-2.5-pro

System: Vertex AI (ADC Enabled)

IBM Legacy Modernization Agent v2.0

Legacy Scribe

1a. Upgrading large-scale legacy systems into intelligent, scalable, future-ready platforms.

Legacy Asset Input

Asset Type

COBOL Source

Paste Code / Logs here:

```
[SCRIBE]: Deconstructing legacy syntax and state machine...
```

Modernization Strategy Generated

System Analysis

Logic Summary: The legacy code establishes a direct database connection to a production database ('DB_PROD') using a hardcoded password embedded within the source code.

Identified Security Vulnerabilities

```
EXEC SQL
CONNECT TO DB_PROD
IDENTIFIED BY
"Admin123!_Secret"
END-EXEC.
```

Initiate Modernization Pipeline

• Hardcoded Credentials: The database password 'Admin123!_Secret' is stored in plain text directly in the source code, posing a severe security risk. This exposes the production database to anyone with access to the code.

• Lack of Credential Rotation: Hardcoded passwords cannot be easily rotated, leading to long-lived, static credentials that increase the window of opportunity for attackers.

Target Architecture: Externalize database credentials from the application code. Use a dedicated secret management service like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault. The application should fetch the database connection string or individual credentials at runtime from this secure, external source. This approach enables centralized management, auditing, and automatic rotation of secrets without requiring code changes.

Refactored Implementation

(Python/FastAPI)

```
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmake

# Retrieve database credentials secure
# These would be set in the deployment
DB_USER = os.getenv("DB_USER")
DB_PASSWORD = os.getenv("DB_PASSWORD")
DB_HOST = os.getenv("DB_HOST", "localh
DB_NAME = os.getenv("DB_NAME", "db_pro
DB_PORT = os.getenv("DB_PORT", "5432")

# Construct the database URL without h
SQLALCHEMY_DATABASE_URL = f"postgresql

# Create the SQLAlchemy engine, which
engine = create_engine(SQLALCHEMY_DATA

# Create a configured "Session" class
SessionLocal = sessionmaker(autocommit

# Example of using the session (e.g.,
def get_db_session():
    db = SessionLocal()
    try:
        print("Database connection suc
        yield db
```

```
finally:
```

Download Modernization Report